

Pensamiento Computacional:

Abstracción:

- ¿Qué se solicita finalmente? (Problema)
R/= Desarrollar un sistema de alquiler de vehículos que permita gestionar: clientes, vehículos, modalidades de alquiler, reservas, y servicios adicionales también manejar tarifas. El registro de los clientes debe de ser con la siguiente información (nombre, documento, teléfono, correo, edad y fecha de registro), el alquiler se registra con los siguientes datos y atributos (código, nombre, descripción, duración mínima en días, valor diario y estado ya sea cualquiera de los siguientes, disponible, suspendido, finalizado). Además, la empresa ofrece modalidades o categorías que son Económica, Ejecutiva y Premium cada una de las anteriores modalidades o categorías ofrece diferentes beneficios Por ejemplo, la PREMIUM requiere info adicional como el tipo de cobertura cantidad de conductores adicionales permitidos y características especiales del servicio. El valor final del costo del alquiler del vehículo depende la modalidad o categoría más los servicios adicionales. Además, cada vehículo debe de contar con los siguientes datos placa, marca, modelo, año, tipo y tarifa diaria estos vehículos se asignan a los clientes que realicen una reserva este vehículo puede ser alquilado por diferentes clientes y además debe de mostrar en cada reserva la relación contratada por el cliente el vehículo y la modalidad. También durante el alquiler el cliente puede solicitar alguno de los servicios adicionales tales como GPS, silla para bebe, conductor adicional o seguro complementario estos tipos de servicios están identificados con un código, nombre, descripción, precio y disponibilidad, además estos servicios se adicionan a la factura final. También debe permitirse que la empresa realice búsquedas sobre sus clientes usando su número de teléfono y ver si ese número es perfecto. También conocer los ingresos en un periodo determinado.
- ¿Qué información es relevante?
R/=
EMPRESA
Nombre de la empresa, dirección, nit, dirección teléfono, correo electrónico y pagina web
CLIENTE
Nombre, documento, teléfono, correo electrónico, edad y fecha de registro
VEHICULO
Placa, marca, modelo, año, tipo y tarifa diaria
SERVICIO ADICIONAL
GPS, silla para bebe, conductor adicional, seguro complementario, código, nombre, descripción, precio y disponibilidad.
SERVICIO

- ¿Cómo se agrupa la información relevante?
R/=
 - CLASE: Empresa
Atributos: nombreComercial, nit, dirección, teléfono, correoElectronico, paginaWeb,
 - CLASE Cliente:
Atributos: nombreCompleto, documentoidentidad, teléfono, correoelectronico, edad, fechaRegistro.
 - CLASE: Vehículo
Atributos: placa, nombre, modelo, año, tipo, tarifaDiaria.
 - CLASE: Servicio
Atributos: código, nombre, descripción, precio, disponibilidad
 - CLASE: Reserva
Atributos: código, nombre, descripción, duración, valorDia, estado
 - Subclase: modalidadEconomica
 - Subclase: modalidadEjecutiva
Atributos: direcciónEntregaVehiculo
 - Subclase: modalidadPremium
Atributos: tipoCobertura, cantidadConductoresAdicionalesPermitidos, característicasEspecialesServicio, modalidadPremium
- ¿Qué funcionalidades se solicitan finalmente?
 Alquilarvehiculo();
 RegistrarVehiculo()
 RegistrarCliente()
 ReservarVehiculo()
 ValorFinalAlquiler()
 ConsultarClientesRegistrados()
 NumPerfecto()
 ConocerIngresosGenerados()

Descomposición:

- ¿Cómo se distribuyen las funcionalidades?
R/=Para cumplir estrictamente con el principio de responsabilidad única (SRP) de SOLID y la arquitectura MVC (MODELO-VISTA-CONTROLADOR) exigida, las funcionalidades se deben de distribuir en capas claras
 - Capa del modelo
Clases de entidad (Cliente, Vehículo, Servicio, Reserva, Alquiler): almacenan los datos de manera limpia mediante encapsulamiento (getters y setters).

- Capa del controlador
Clases controller (javaFX) capturan los eventos los clicks etc., capturan de forma superficial los campos de la interfaz de usuario y delegan la ejecución real hacia la capa del modelo
- Capa de la vista (User Interface)
Es la capa que contiene todos los archivos FXML/JavaFX construyen la interfaz gráfica de usuario para el personal encargado de la administración del sistema, no contiene lógica de negocio, solo componentes gráficos (botones, tablas, formularios)
- ¿Qué debo de hacer para probar las funcionalidades?
R/=

Para garantizar el correcto funcionamiento del sistema sin mezclar la interfaz gráfica con el motor del programa, se deben seguir estos pasos secuenciales de prueba:

- Pruebas Unitarias de Lógica Pura (Consola / JUnit):
- Instanciar clientes de prueba y verificar el método de detección de números perfectos enviando números conocidos (como el 6 o el 28) y números comunes para asegurar que la lógica matemática responda de forma correcta.
- Crear una reserva simulada enlazando un vehículo, una modalidad y dos servicios adicionales; imprimir en consola el resultado de calcularValorFinal() y verificar manualmente con calculadora que el total coincida con las tarifas sumadas.
- Pruebas de Flujo en Interfaz Gráfica (JavaFX):
- Digitar datos incompletos en el formulario de registro de clientes para constatar que la vista lance alertas de validación.
- Hacer clic en el botón de búsqueda por teléfono y constatar en pantalla si el sistema notifica visualmente si el usuario cumple con la condición de número perfecto.
- Filtrar y consultar los ingresos totales generados en un rango de fechas específico, verificando que la tabla se actualice y muestre la sumatoria correcta acumulada.

Reconocimiento de patrones:

- ¿Qué puedo reutilizar de la solución de otros problemas?

1. Reutilización de Patrones de Diseño (Arquitectura)

- El Patrón Singleton (Instancias Únicas): Las clases RentCar, RentCarRepository, ClienteService, AlquilerService y FinanzasService ya tienen implementado el método public static synchronized ... getInstancia(). Si el enunciado nos pide crear un nuevo módulo (por ejemplo, un MantenimientoService para los carros), **puedo copiar exactamente esta estructura de constructor privado e instancia estática para asegurar que no se dupliquen los datos en memoria.**
- El Patrón Builder (Creación de Objetos Complejos): La clase interna ClienteBuilder nos sirve como plantilla exacta. Si el enunciado me exige añadir validaciones obligatorias al registrar un carro (por ejemplo, que la placa tenga un formato específico), **podemos implementar un VehiculoBuilder reutilizando la misma lógica del de clientes.**

2. Reutilización de Capas de Lógica de Negocio (Servicios)

- AlquilerService y LiquidadorFinanciero: El método generarFacturaAlquiler y la clase LiquidadorFinanciero ya calculan tarifas base de vehículos y suman los precios de ServicioAdicional. Si el enunciado nos pidiera meter promociones (ej. *"si alquila más de 7 días, descuento un 10%"*), no tenemos que reescribir todo; solo debería entrar a LiquidadorFinanciero y agregar esa condición matemática al algoritmo que ya calcula el total.
- FinanzasService: Este servicio ya contiene la lógica para filtrar y buscar reservas en un rango de fechas (calcularIngresosPeriodo). Sirve como base exacta para cualquier otra consulta estadística que vayan a pedir (como *"saber cuál fue el vehículo más alquilado en un mes"*), reutilizando los ciclos for y los métodos isAfter() o isBefore() de LocalDate.

3. Reutilización de Polimorfismo (Estructura de Clases)

- ModalidadAlquiler y sus hijas (Económica, Ejecutiva, Premium): El método abstracto determinarLogísticaEntrega() es ejemplo de polimorfismo que usamos. Si el enunciado nos pide agregar una nueva categoría de alquiler (por ejemplo, *"Modalidad Empresarial con chofer"*), no tendríamos que modificar las clases viejas. Solo creamos una nueva clase llamada ModalidadEmpresarial que extienda (extends) de ModalidadAlquiler y sobrescribiría el método para definir su propia logística. Esto cumple a la perfección con el principio Abierto/Cerrado de SOLID.

4. Capa de Datos e Inversión de Dependencias (DIP)

- RentCarRepository y RentCarRepository: Todo el almacenamiento en memoria con colecciones List<> ya funciona. Si el parcial nos pide agregar un CRUD completo para una nueva entidad (como registrar los empleados de la empresa), solo añadimos los métodos guardar Empleado() y obtenerEmpleados() dentro de la interfaz y su repositorio, reutilizando la base de datos simulada en memoria que ya tenemos construida.

Codificación:

- ¿Cómo pruebo la solución en JAVA?

Para probar la solución a la que el equipo de desarrollo ha llegado se han implementado pruebas unitarias en JUnit

¿Cómo escribo la solución en JAVA?

R/= Desglose Arquitectónico del Modelo

- Patrón Singleton (Instancia Única): Está implementado de forma sincronizada en los componentes del sistema: la entidad de datos unificada RentCar, la persistencia en memoria RentCarRepository, y los módulos de lógica de negocio ClienteService, AlquilerService y FinanzasService.
- Patrón Creacional Builder: La clase interna estática ClienteBuilder encapsula la lógica de validación de los datos mandatorios (nombre y documento), aislando la instanciación de objetos de tipo Cliente de mutaciones o inconsistencias externas.

- Polimorfismo Completo (Principio LSP - SOLID): La estructura abstracta ModalidadAlquiler define el método abstracto contractual determinarLogisticaEntrega(). Este es sobrescrito por las modalidades concreta ModalidadEconomica (reclamación en sede), ModalidadEjecutiva (dirección personalizada) y ModalidadPremium (cobertura VIP total).
- Inversión de Dependencias (Principio DIP): Las capas de servicios de aplicación interactúan de manera desacoplada con el repositorio apoyándose en la abstracción pura provista por la interfaz IRentCarRepository, impidiendo el acoplamiento rígido hacia la clase concreta de datos en memoria.